

BÁO CÁO DỰ ÁN: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Online Auction System — Group 7 (CS6)

Môn học: Lập trình Nâng cao | Trường Đại học Công nghệ (UET - VNU)

1. Giới thiệu mục tiêu và phạm vi thực hiện

1.1. Mục tiêu

Xây dựng một hệ thống đấu giá trực tuyến hoàn chỉnh theo mô hình **Client-Server**, cho phép nhiều người dùng cùng tham gia đấu giá sản phẩm theo thời gian thực. Hệ thống hướng tới các mục tiêu cụ thể:

- Cung cấp nền tảng đấu giá công bằng, minh bạch với cập nhật giá **realtime** qua TCP Socket.
- Đảm bảo **tính toàn vẹn dữ liệu** khi nhiều người đặt giá đồng thời (concurrent bidding).
- Tích hợp hệ thống ví điện tử với cơ chế đóng băng số dư, đảm bảo an toàn giao dịch.
- Xây dựng giao diện người dùng JavaFX hiện đại, trực quan và dễ sử dụng.

1.2. Phạm vi thực hiện

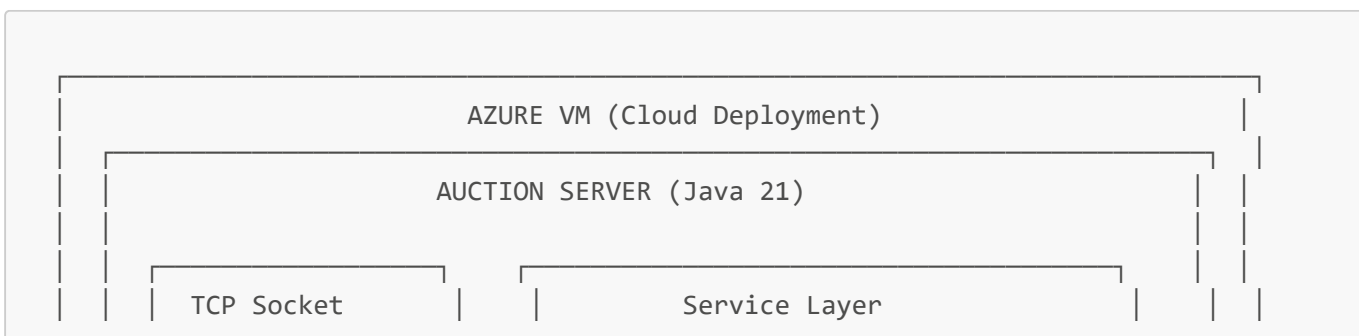
Hệ thống hỗ trợ **3 vai trò** với luồng nghiệp vụ khép kín:

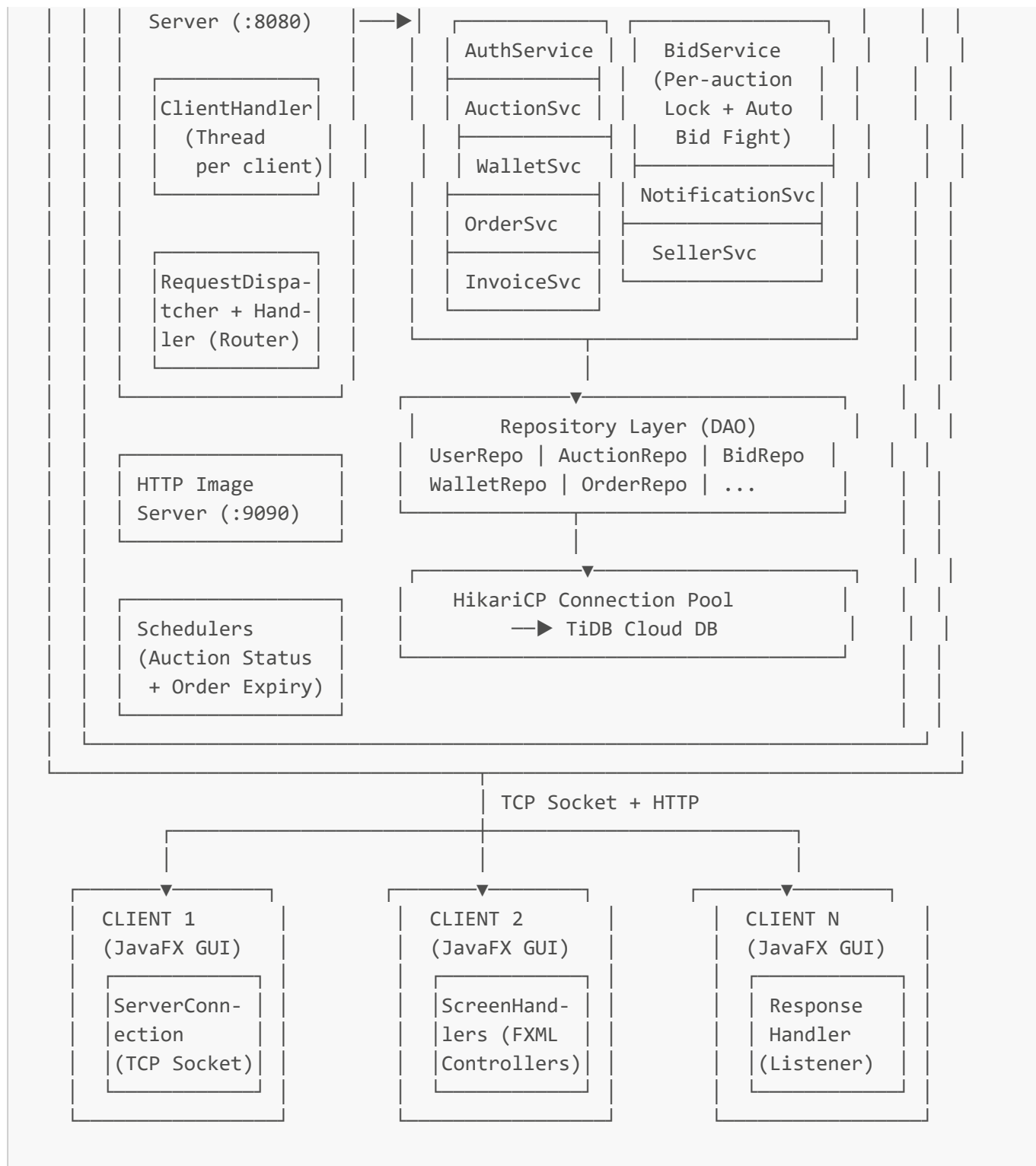
Vai trò	Phạm vi chức năng chính
Bidder (Người mua)	Đăng ký/Đăng nhập, đấu giá realtime, cấu hình auto-bid, quản lý ví điện tử, thanh toán đơn hàng, nhận thông báo
Seller (Người bán)	Đăng sản phẩm với thuộc tính động, theo dõi phiên đấu giá, quản lý đơn hàng, xuất hóa đơn PDF
Admin (Quản trị viên)	Quản lý người dùng, phê duyệt seller/giao dịch/sản phẩm, giám sát hệ thống

Môi trường triển khai: Server được vận hành 24/7 trên máy ảo **Azure VM**, cơ sở dữ liệu cloud **TiDB** (Alibaba Cloud), pipeline CI/CD qua **GitHub Actions** với auto-deploy.

2. Kiến trúc tổng thể của hệ thống

2.1. Sơ đồ kiến trúc





2.2. Mô tả kiến trúc

Hệ thống được xây dựng theo mô hình **Multi-module Maven** gồm 3 module chính:

① Auction_shared (Module dùng chung)

- Chứa các lớp Model (29 lớp), Enum (10 loại), Request DTOs (36 loại), Response DTOs (41 loại), và NetworkConfig.
- Được cả Server và Client depend vào, đảm bảo giao tiếp thống nhất kiểu dữ liệu.
- Giao thức truyền thông: Sử dụng **Java Object Serialization** (`ObjectOutputStream` / `ObjectInputStream`) qua TCP Socket. Client gửi **RequestDTO**, Server xử lý và trả về **ResponseDTO** — tất cả đều implement `Serializable`.

② Auction_server (Module Backend)

- **ServerController Layer:** `Server.java` khởi tạo `ServerSocket` trên cổng 8080 với `Executors.newCachedThreadPool()` — tự động tạo thread cho mỗi client kết nối. `ClientHandler` sử dụng **Java 21 switch pattern matching** (`switch(requestObj)` với case `SignUpRequestDTO req - >`) để điều hướng 35+ loại request đến `RequestHandler` → `RequestDispatcher` → Service tương ứng.
- **Service Layer:** Xử lý toàn bộ business logic (10 service: `BidService`, `WalletService`, `AuthService`, `OrderService`, `NotificationService`...). Mỗi service áp dụng pattern **Singleton (Bill Pugh Holder idiom)** với dual constructor — production dùng `getInstance()`, test dùng constructor có tham số để inject mock.
- **Repository Layer:** 9 Repository thao tác trực tiếp với database qua JDBC thuần, sử dụng **HikariCP** connection pool (min 5, max 20 connections, keepalive 60s) kết nối **TiDB Cloud**. Hỗ trợ `SELECT ... FOR UPDATE` cho row-level locking.
- **Scheduler:** Hai tiến trình ngầm chạy mỗi **5 giây** qua `ScheduledExecutorService`:
 - `AuctionStatusScheduler`: (1) **Tự động mở phiên** — quét các phiên trạng thái `WAITING` có `startTime ≤ now` → chuyển sang `ACTIVE` và broadcast `AuctionStatusUpdatedDTO` đến tất cả client; (2) **Tự động đóng phiên** — quét các phiên `ACTIVE` có `endTime ≤ now` → chuyển sang `CLOSED` (dưới per-auction lock để tránh race condition với bid), xác định winner, tạo Order và gửi notification.
 - `OrderExpiryScheduler`: Tự động hủy đơn hàng quá hạn 7 ngày chưa thanh toán.
- **ImageHttpServer:** HTTP server nhẹ phục vụ ảnh sản phẩm trên cổng 9090, có bảo vệ chống path traversal.

③ Auction_client (Module Frontend)

- **ServerConnection:** Quản lý kết nối TCP Socket đến Server (static singleton, auto-reconnect khi mất kết nối). Spawns một **daemon listener thread** chạy nền liên tục đọc response.
- **ResponseHandler** (939 dòng): Listener dispatch response về đúng callback handler, sử dụng `Platform.runLater()` để cập nhật UI an toàn trên JavaFX Application Thread.
- **ScreenHandlers: 30 FXML Controller** (25 user/bidder/seller + 5 admin) quản lý **22 màn hình** giao diện (Login, Register, Homepage, AuctionRoom, Wallet, Profile, Seller Dashboard, Admin panels...).

3. Các chức năng đạt được

3.1. Xác thực & Quản lý Tài khoản

Chức năng	Hướng giải quyết	Lý do lựa chọn
Đăng ký / Đăng nhập	Mật khẩu được băm bằng bcrypt trước khi lưu DB. Validate email/username trùng lặp ở server-side.	BCrypt là thuật toán băm one-way với salt ngẫu nhiên, chống brute-force hiệu quả. Server-side validation tránh bypass từ client.
Single Session	Server duy trì <code>ConcurrentHashMap<userId, ClientHandler></code> — khi user đăng nhập mới, phiên cũ bị	Đảm bảo consistency dữ liệu: tránh 2 phiên cùng thao tác trên 1 ví/tài khoản gây xung đột.

Chức năng	Hướng giải quyết	Lý do lựa chọn
	kick ngay lập tức. Xử lý client đóng đột ngột qua sự kiện <code>onDisconnect</code> .	
Hồ sơ cá nhân	Cập nhật thông tin qua DTO → Service validate → Repository update. Ảnh đại diện upload lên HTTP Image Server.	Tách biệt rõ ràng lớp xử lý, dễ bảo trì và mở rộng.

3.2. Đấu giá thời gian thực (Core Feature)

Chức năng	Hướng giải quyết	Lý do lựa chọn
Phòng đấu giá Realtime	Mỗi phiên đấu giá được liên kết với một <code>AuctionRoom</code> trên Server để quản lý danh sách các <code>ClientHandler</code> đang xem. Khi có bất kỳ sự thay đổi nào (đặt giá mới, cập nhật thời gian), Server thực hiện broadcast <code>NewBidDTO</code> hoặc <code>AuctionPriceUpdateDTO</code> đến tất cả Client trong phòng và trang chủ để cập nhật UI.	Room-based broadcast giảm tối đa băng thông truyền tải mạng: chỉ gửi dữ liệu cập nhật cho các Client thực sự quan tâm hoặc đang theo dõi phiên đấu giá đó, tránh broadcast vô tội vạ toàn hệ thống.
Đặt giá thủ công (Manual Bid)	Người dùng đặt giá trực tiếp thông qua form trong phòng đấu giá. Giá đặt mới phải thỏa mãn: $giá_mới \geq giá_hiện_tại + bước_giá_tối_thiểu$ (<code>minStepPrice</code>). Hệ thống kiểm tra số dư ví khả dụng của người dùng phải đủ $\geq 10\%$ giá trị đặt giá mới để đóng băng làm tiền đặt cọc trước khi chấp nhận lượt bid.	Ràng buộc giá trị đặt cọc tối thiểu (10%) trực tiếp trên luồng đặt giá giúp loại bỏ hoàn toàn các lượt bid ảo, phá giá từ các tài khoản không có khả năng tài chính thực tế.
Đấu giá tự động (Auto-Bid)	Người dùng thiết lập một mức giá tối đa (<code>maxPrice</code>). Khi có bidder khác đặt giá cao hơn, Server chạy thuật toán State-Machine loop trực tiếp trên RAM: tự động tăng giá thầu của người dùng theo bước giá tối thiểu cho đến khi đạt <code>maxPrice</code> . Toàn bộ quá trình tranh đấu giá tự động (Auto-Bid Fight) được kiểm soát bởi per-auction lock và áp dụng mô hình PostCommitEvents (gồm toàn bộ sự kiện truyền thông để phát sóng một lần duy nhất sau khi DB hoàn tất việc commit).	Tối ưu hóa hiệu năng hệ thống bằng cách thực hiện toàn bộ luồng so sánh, loại bỏ bot đấu giá tự động trực tiếp trên bộ nhớ (RAM), sau đó mới ghi nhận một lần duy nhất vào CSDL và phát sóng sự kiện, tránh nghẽn luồng I/O database.
Cơ chế chống bắn tia (Anti-Sniping)	Khi một lượt đặt giá hợp lệ (thủ công hoặc tự động) được ghi nhận trong vòng 3 phút cuối của phiên đấu giá, hệ thống sẽ tự động kéo dài thời gian kết thúc phiên đấu giá thêm đúng 3 phút từ thời điểm đặt giá đó. Thời gian cập nhật sẽ được lưu xuống DB và phát sóng realtime về các Client.	Ngăn chặn chiến thuật "sniping" (đặt giá thầu vào giây cuối cùng để người khác không kịp phản ứng), tạo môi trường đấu giá lành mạnh, công bằng và

Chức năng	Hướng giải quyết	Lý do lựa chọn
		tối đa hóa giá trị sản phẩm cho người bán.
Xử lý Đặt giá đồng thời (Concurrent Bidding)	Sử dụng cơ chế đồng bộ hóa luồng (Thread-safety) bằng per-auction <code>synchronized</code> lock thông qua <code>ConcurrentHashMap<auctionId, Object></code> trên Server, kết hợp row-level locking (<code>SELECT ... FOR UPDATE</code> trên SQL) đối với dữ liệu phiên đấu giá và số dư ví của các bidder.	Bảo vệ toàn vẹn dữ liệu (data consistency) khi có hàng ngàn người dùng cùng đặt giá tại cùng một mili-giây. Đảm bảo các lượt bid được xử lý tuần tự, chính xác trong cùng một phiên, nhưng song song hoàn toàn giữa các phiên đấu giá khác nhau để không gây nghẽn cổ chai hệ thống.
Đồng bộ hóa thời gian đếm ngược	Tránh việc Client và Server bị lệch múi giờ hoặc lệch giây do độ trễ truyền dẫn mạng. Client sử dụng JavaFX Timeline đếm ngược cục bộ dựa trên timestamp kết thúc (<code>endTime</code>) cố định được trả về từ Server, thay vì liên tục gửi request truy vấn thời gian còn lại.	Giảm thiểu tối đa số lượng request không cần thiết gửi lên Server, đồng thời đảm bảo thời gian đếm ngược trên giao diện của người dùng chạy cực kỳ mượt mà, không bị giật lag.

3.3. Ví điện tử & Giao dịch

Chức năng	Hướng giải quyết	Lý do lựa chọn
Balance Freeze/Release	Khi bid thắng → đóng băng 10% giá trị bid làm cọc (<code>freezeMoney</code>). Khi bị outbid → tự động hoàn trả (<code>releaseFrozen</code>). Thanh toán → chuyển tiền buyer→seller (<code>processPayment</code>). Tất cả dùng DB transaction + row-level lock trên wallet.	Cơ chế cọc đảm bảo bidder có khả năng thanh toán, tránh bid ảo. Row-level lock ngăn race condition trên số dư ví.
Yêu cầu Nạp tiền	Người dùng gửi yêu cầu nạp tiền, hệ thống lưu <code>WalletTransaction</code> trạng thái <code>PENDING</code> . Khi Admin duyệt yêu cầu → gọi <code>WalletService.processTransactionRequest()</code> để chuyển trạng thái thành <code>APPROVED</code> và cộng số dư khả dụng (<code>wallet.deposit()</code>).	Đảm bảo tính chính xác của tài chính. Admin đối soát giao dịch thực tế qua ngân hàng trước khi xác nhận cộng tiền hệ thống để tránh nạp khống.
Yêu cầu Rút tiền	Khi người dùng tạo yêu cầu rút tiền, hệ thống lập tức thực hiện khấu trừ trực tiếp số tiền rút khỏi số dư khả dụng	Tránh lỗi thâm hụt tài khoản (overdraft).

Chức năng	Hướng giải quyết	Lý do lựa chọn
	(<code>wallet.withdraw()</code>) và tạo giao dịch PENDING . Khi Admin duyệt → chuyển trạng thái giao dịch thành APPROVED (không trừ tiền nữa). Nếu Admin từ chối (REJECTED) → hoàn trả lại số tiền vào ví của người dùng (<code>wallet.deposit()</code>).	Nếu không khấu trừ ngay mà đợi Admin duyệt mới trừ, người dùng có thể chi tiêu/đấu giá hết số tiền đó trước khi Admin duyệt. Khi Admin duyệt sau đó, ví sẽ không còn đủ số dư để khấu trừ thực tế, gây lỗi logic hệ thống.

3.4. Quản lý Đơn hàng & Hóa đơn

Chức năng	Hướng giải quyết	Lý do lựa chọn
Tự động mở phiên đấu giá	<code>AuctionStatusScheduler</code> quét mỗi 5s → phiên WAITING có <code>startTime ≤ now</code> → chuyển sang ACTIVE + broadcast <code>AuctionStatusUpdateDTO</code> đến tất cả client để cập nhật giao diện realtime.	Cho phép Seller đặt lịch mở phiên đấu giá trong tương lai mà không cần thao tác thủ công. Scheduler đảm bảo phiên được kích hoạt đúng thời điểm.
Tự động đóng phiên & tạo đơn hàng	Scheduler quét phiên ACTIVE có <code>endTime ≤ now</code> → dưới per-auction lock chuyển sang CLOSED → xác định winner → tạo Order trạng thái PENDING + gửi notification cho winner và seller. Nếu không có ai bid → hủy phiên, thông báo cho seller.	Đóng phiên dưới auction lock đảm bảo không race condition với bid đang xử lý. Tạo đơn hàng tự động giúp luồng nghiệp vụ liền mạch.
Hủy đơn quá hạn	<code>OrderExpiryScheduler</code> tự động hủy đơn >7 ngày chưa thanh toán, áp dụng penalty (buyer mất cọc → chuyển cho seller).	Đảm bảo hệ thống không tồn đọng đơn hàng treo, bảo vệ quyền lợi seller.
Xuất hóa đơn PDF	Dùng <code>OpenHTMLtoPDF</code> render từ HTML template → PDF. Hỗ trợ Unicode tiếng Việt qua embedded Arial font.	HTML template dễ thiết kế giao diện hóa đơn, dễ bảo trì hơn so với tạo PDF bằng code thuần.

3.5. Hệ thống Thông báo

Chức năng	Hướng giải quyết	Lý do lựa chọn
Notification Realtime	<code>NotificationService</code> vừa lưu DB vừa gửi realtime qua <code>Server.sendToUser()</code> . Template hoá 17+ kịch bản thông báo trong <code>NotificationTemplate</code> .	Lưu DB đảm bảo không mất thông báo khi user offline. Gửi

Chức năng	Hướng giải quyết	Lý do lựa chọn
		realtime qua socket giúp trải nghiệm tức thời.
Toast Notification UI	Custom JavaFX popup với animation slide-in/slide-out, auto-hide sau 8s, click callback điều hướng.	UX hiện đại, không gây gián đoạn thao tác người dùng nhưng vẫn thu hút chú ý.

3.6. Admin Dashboard

Chức năng	Hướng giải quyết	Lý do lựa chọn
Quản lý tài khoản User	Admin xem danh sách người dùng và trạng thái tài khoản. Khi Admin thực hiện khóa (lock) tài khoản qua UserManagerController , hệ thống cập nhật DB và ngay lập tức tìm kiếm ClientHandler online tương ứng để ngắt kết nối (kick) khỏi hệ thống.	Ngăn chặn tức thời các tài khoản vi phạm chính sách hoặc có dấu hiệu gian lận mà không cần đợi phiên làm việc của họ tự hết hạn.
Phê duyệt vai trò Seller	Quản lý danh sách các yêu cầu nâng cấp lên Seller qua SellerAccountManagerController . Admin có quyền phê duyệt (APPROVED) hoặc từ chối (REJECTED). Khi duyệt, vai trò được đổi sang SELLER và gửi thông báo realtime kết quả về Client.	Kiểm soát chất lượng đối tác bán hàng, hạn chế sản phẩm rác hoặc lừa đảo trên hệ thống.
Phê duyệt giao dịch ví	Admin xem danh sách giao dịch PENDING qua PendingTransactionManagerController để duyệt nạp/rút tiền. Thực hiện duyệt/từ chối giao dịch: duyệt nạp/rút để hoàn tất giao dịch; từ chối rút để kích hoạt hoàn lại tiền cho ví người dùng.	Kiểm soát chặt chẽ dòng tiền, hỗ trợ kiểm soát thủ công trước khi phê duyệt giao dịch thực tế, giảm thiểu rủi ro lỗi số dư hoặc thất thoát.
Kiểm duyệt phiên đấu giá	Duyệt/Từ chối sản phẩm đấu giá mới do Seller tạo ở trạng thái PENDING_APPROVAL qua AuctionManagerController . Nếu được duyệt, phiên chuyển sang WAITING (chờ scheduler tự động mở). Nếu từ chối, chuyển sang REJECTED kèm lý do.	Đảm bảo tính hợp lệ của sản phẩm đấu giá (hình ảnh, giá khởi điểm, thuộc tính động) trước khi công khai tới toàn bộ người mua.

3.7. Tạo phiên đấu giá & Thuộc tính sản phẩm động (Seller Feature)

Chức năng	Hướng giải quyết	Lý do lựa chọn
Đăng tải sản phẩm mới	Seller đăng sản phẩm mới kèm thông tin (tên, giá khởi điểm, bước giá tối thiểu, ảnh, thời gian bắt đầu/kết thúc) và thuộc tính động theo danh mục (ItemType). Sử dụng Factory Method Pattern (ItemFactory) để sinh đối tượng tương ứng (ElectronicItem , VehicleItem ...) và gửi qua UploadItemRequestDTO .	Cho phép hỗ trợ nhiều danh mục sản phẩm khác nhau (điện tử, xe cộ, nghệ thuật...) với các thuộc tính đặc thù riêng biệt mà không làm phình to bảng cơ sở dữ liệu hay cấu trúc code.
Vòng đời phiên đấu giá	Phiên đấu giá chuyển trạng thái tuần tự: PENDING_APPROVAL (sau khi tạo) -> WAITING (Admin duyệt xong) -> ACTIVE (được scheduler mở khi đến giờ) -> CLOSED (kết thúc phiên, tự động chốt người thắng và tạo đơn hàng).	Đảm bảo luồng nghiệp vụ tự động hóa cao, chính xác và đồng bộ theo thời gian thực nhưng vẫn đảm bảo tính kiểm duyệt của quản trị viên.

3.8. Tính năng kỹ thuật nâng cao

Chức năng	Hướng giải quyết	Lý do lựa chọn
CI/CD	2 GitHub Actions workflow: CI (build + test tự động với xvfb-run cho JavaFX headless + JaCoCo coverage + publish test results) và CD (auto SSH deploy lên Azure VM khi push branch develop).	CI phát hiện lỗi sớm trên mỗi commit. CD tự động hoá deploy, giảm thiểu lỗi thủ công.
Unit Testing	12 file test với JUnit 5 + Mockito . Service dùng Dependency Injection (dual constructor: production singleton + test-injectable). JaCoCo aggregate coverage qua module Auction_report .	DI pattern cho phép mock hoàn toàn repository layer, test business logic độc lập không cần DB thật.
Thread-safe Socket I/O	synchronized (out) trên output stream của mỗi ClientHandler (35+ synchronized blocks). ServerConnection phía client cũng synchronized trên sendData() .	Ngăn chặn interleaved writes khi nhiều thread (service, scheduler, broadcast) cùng gửi response cho 1 client.

Báo cáo được thực hiện bởi Group 7 — CS6, Trường Đại học Công nghệ (UET - VNU).